MNR UNIVERSITY

School of Engineering & Technology and AI

Machine Learning Lab Record

Name Yashwant Talari

Roll_no 25RBAIM0047

Section A

Year and Semester: 2-1

Branch: B. Tech (Hons.) / CSE(AI&DS)

Course code: 25CSM201

Course name: Machine Learning

Submitted To: Dr M Praveen

LAB01

Date:23/07/2026

Title of the Lab: Introduction to Python Programming and Machine Learning Environment Setup.

Aim: To familiarize students with Python programming, install the required Machine Learning libraries, set up the Machine Learning development environment, and execute a simple Python program for Machine Learning.

Software Requirements

- Python 3.12 or above
- Jupyter Notebook / Jupyter Lab
- VS Code (Optional)
- Google Collab
- pip Package Manager

Python Libraries

- NumPy
- Pandas
- Matplotlib
- Scikit-learn

Theory: Python is a high-level, interpreted, and object-oriented programming language known for its simple syntax and readability. It is widely used in Machine Learning because it enables fast development and has a large collection of powerful libraries. Popular libraries include NumPy for numerical computing, Pandas for data manipulation, Matplotlib for visualization, and Scikit-learn for Machine Learning algorithms. Machine Learning development is commonly performed using Jupyter Notebook or Google Collab, which provide interactive environments for writing and executing Python code. Jupyter Notebook supports code, visualizations, and documentation in one place, while Google Collab offers a cloud-based platform with free GPU access and easy

collaboration. These tools make Python an ideal choice for Machine Learning applications.

Algorithm:

- Install Python.
- Install Jupyter Notebook.
- Install required Machine Learning libraries.
- Open Jupyter Notebook.
- Create a new notebook.
- Import required libraries.
- Execute a simple Python program.
- Verify successful execution.

#Python Program:

[1]:
```
#LAB 01
#Introduction to Python Programming and Machine Learning Environment Setup

# Printing a message
print("Welcome to Machine Learning Laboratory")

# Import Machine Learning Libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import sklearn

print("\nLibraries Imported Successfully!")

# Display Library Versions
print("\nNumPy Version :", np.__version__)
print("Pandas Version :", pd.__version__)
print("Scikit-Learn Version :", sklearn.__version__)

# Simple NumPy Array
numbers = np.array([10, 20, 30, 40, 50])

print("\nNumPy Array")
print(numbers)

# Create a Pandas DataFrame
student = {
    "Roll No":[1,2,3],
    "Name":["Rahul","Anita","Kiran"],
    "Marks":[85,92,78]
}

df = pd.DataFrame(student)

print("\nStudent Data")
print(df)
```

Output:

```
Welcome to Machine Learning Laboratory

Libraries Imported Successfully!

NumPy Version : 2.3.5
Pandas Version : 2.3.3
Scikit-Learn Version : 1.7.2

NumPy Array
[10 20 30 40 50]

Student Data
   Roll No   Name  Marks
0        1  Rahul     85
1        2  Anita     92
2        3  Kiran     78
```

Result: The Python programming environment and required Machine Learning libraries were successfully installed and verified. A basic Python program using NumPy and Pandas was executed successfully.

LAB02

Date:23/07/2026

Title of the Lab: Working with NumPy and Pandas for Data Manipulation.

Aim: To understand and perform data manipulation using NumPy and Pandas libraries in Python.

Software Requirements

- Python 3.x
- Jupyter Notebook / Google Collab
- NumPy
- Pandas

Theory: NumPy and Pandas are two essential Python libraries widely used in data science and machine learning. **NumPy** focuses on numerical computing, offering high-performance multidimensional arrays, matrix operations, random number generation, and efficient memory usage, making it faster than traditional Python lists. **Pandas**, on the other hand, is designed for data analysis and manipulation, providing powerful structures like **Series** (one-dimensional labeled arrays) and **Data Frames** (two-dimensional tabular data). With features such as data cleaning, filtering, sorting, handling missing values, aggregation, and statistical analysis, Pandas simplifies working with structured datasets. For example, a NumPy array might look like `[10,20,30,40,50]`, while a Pandas Series adds labels to values, and a Data Frame organizes data into rows and columns such as Roll No, Name, and Marks. Together, NumPy and Pandas form the backbone of machine learning workflows, supporting tasks like data collection, cleaning, transformation, analysis, feature selection, and preprocessing—ensuring datasets are ready for algorithms to deliver accurate results.

Algorithm

1. Import NumPy and Pandas libraries.
2. Create a NumPy array.
3. Perform arithmetic operations on the array.
4. Create a Pandas Data Frame.
5. Display the dataset.
6. Perform basic statistical analysis.
7. Filter data based on a condition.
8. Display the final results.

#Python Program

[2]:
```
#LAB 02
#Working with NumPy and Pandas for Data Manipulation
# Experiment 2
# Working with NumPy and Pandas for Data Manipulation

import numpy as np
import pandas as pd

# ----------------------------
# NumPy Operations
# ----------------------------

arr = np.array([10, 20, 30, 40, 50])

print("NumPy Array")
print(arr)

print("\nArray + 5")
print(arr + 5)

print("\nArray * 2")
print(arr * 2)

print("\nMean =", np.mean(arr))
print("Maximum =", np.max(arr))
print("Minimum =", np.min(arr))
```

```python
# ------------------------------
# Pandas Operations
# ------------------------------

data = {
    "Roll No":[101,102,103,104,105],
    "Name":["Rahul","Anita","Kiran","Priya","Ravi"],
    "Marks":[85,92,78,95,88]
}

df = pd.DataFrame(data)

print("\nStudent Data")
print(df)

print("\nDataset Shape")
print(df.shape)

print("\nSummary Statistics")
print(df.describe())

print("\nStudents Scoring More Than 85 Marks")
print(df[df["Marks"] > 85])
```

#Output

```
NumPy Array
[10 20 30 40 50]

Array + 5
[15 25 35 45 55]

Array * 2
[ 20  40  60  80 100]

Mean = 30.0
Maximum = 50
Minimum = 10

Student Data
   Roll No   Name  Marks
0      101  Rahul     85
1      102  Anita     92
2      103  Kiran     78
3      104  Priya     95
4      105   Ravi     88

Dataset Shape
(5, 3)

Summary Statistics
          Roll No      Marks
count    5.000000   5.000000
mean   103.000000  87.600000
std      1.581139   6.580274
min    101.000000  78.000000
25%    102.000000  85.000000
50%    103.000000  88.000000
75%    104.000000  92.000000
max    105.000000  95.000000

Students Scoring More Than 85 Marks
   Roll No   Name  Marks
1      102  Anita     92
3      104  Priya     95
4      105   Ravi     88
```

Result

The experiment was successfully executed. NumPy was used to perform numerical computations, and Pandas was used to create, manipulate, analyze, and filter tabular data. These libraries form the foundation for data preprocessing in Machine Learning.

LAB03

Date:

Title of the Lab: Data Visualization using Matplotlib (Exploratory Data Analysis).

Aim: To perform **Exploratory Data Analysis (EDA)** using the **Matplotlib** library by creating different types of graphs for understanding and visualizing data.

Software Requirements

- Python 3.x
- Jupyter Notebook / Google Colab
- NumPy
- Pandas
- Matplotlib
- Scikit-learn

Theory: Exploratory Data Analysis (EDA) is the process of examining, summarizing, and visualizing datasets before applying Machine Learning algorithms. It helps identify trends, patterns, outliers, and relationships among variables, ensuring the data is suitable for model training. EDA is an essential step in the Machine Learning life cycle because it improves data quality and assists in selecting appropriate algorithms. Data visualization transforms numerical data into graphical form, making it easier to understand. Matplotlib is a Python library used for creating static, animated, and interactive visualizations. It provides various plotting functions for analyzing datasets.

Algorithm

1. Import the required libraries.
2. Load the Iris dataset.
3. Convert the dataset into a Pandas Data Frame.
4. Display the first five records.
5. Create different types of plots using Matplotlib.
6. Observe and interpret the graphs.
7. Analyze the data patterns.

```python
#LAB 03
#Data Visualization using Matplotlib (Exploratory Data Analysis)

import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris

# Load Iris Dataset
iris = load_iris()

# Create DataFrame
df = pd.DataFrame(iris.data, columns=iris.feature_names)

df["Species"] = iris.target

print("First Five Records")
print(df.head())

# ----------------------------
# Line Plot
# ----------------------------
plt.figure(figsize=(6,4))
plt.plot(df["sepal length (cm)"])
plt.title("Line Plot - Sepal Length")
plt.xlabel("Sample")
plt.ylabel("Sepal Length")
plt.show()

# ----------------------------
# Bar Chart
# ----------------------------
species_count = df["Species"].value_counts()

plt.figure(figsize=(5,4))
plt.bar(species_count.index.astype(str), species_count.values)
plt.title("Bar Chart - Species Count")
plt.xlabel("Species")
plt.ylabel("Count")
plt.show()
```

```python
# ----------------------------
# Histogram
# ----------------------------
plt.figure(figsize=(6,4))
plt.hist(df["petal length (cm)"], bins=10)
plt.title("Histogram - Petal Length")
plt.xlabel("Petal Length")
plt.ylabel("Frequency")
plt.show()

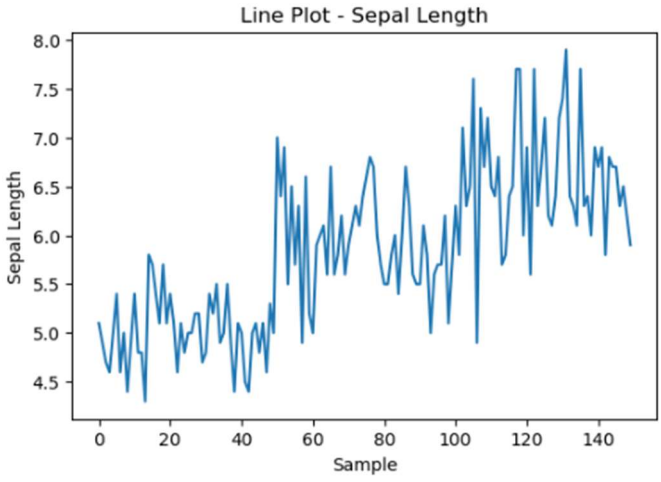
# ----------------------------
# Scatter Plot
# ----------------------------
plt.figure(figsize=(6,4))
plt.scatter(df["sepal length (cm)"], df["petal length (cm)"])
plt.title("Scatter Plot")
plt.xlabel("Sepal Length")
plt.ylabel("Petal Length")
plt.show()

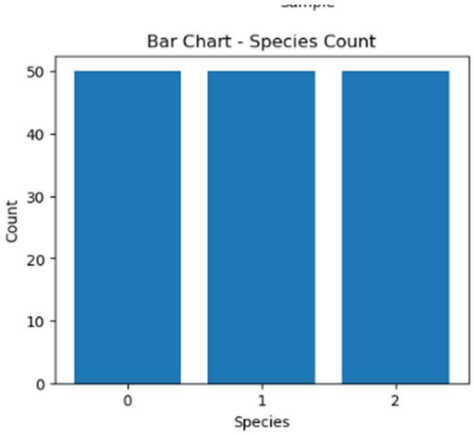
# ----------------------------
# Pie Chart
# ----------------------------
plt.figure(figsize=(5,5))
plt.pie(
    species_count.values,
    labels=["Setosa","Versicolor","Virginica"],
    autopct="%1.1f%%"
)
plt.title("Species Distribution")
plt.show()
```

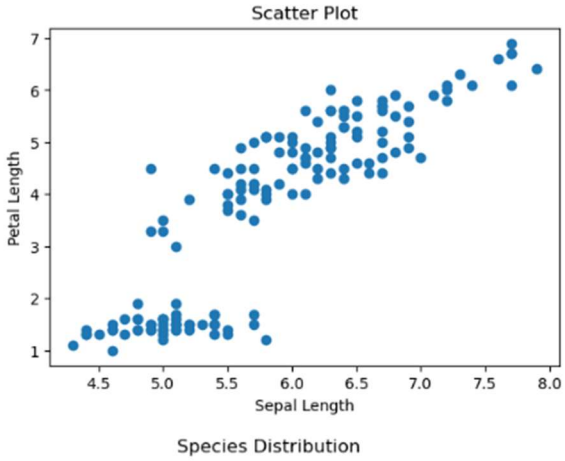
OUTPUT

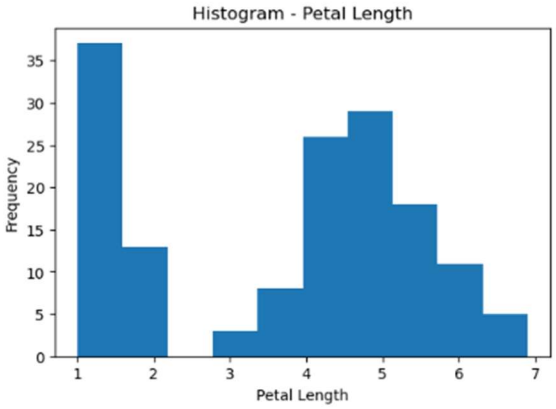
```
First Five Records
   sepal length (cm)  sepal width (cm)  petal length (cm)  petal width (cm)  \
0                5.1               3.5                1.4               0.2
1                4.9               3.0                1.4               0.2
2                4.7               3.2                1.3               0.2
3                4.6               3.1                1.5               0.2
4                5.0               3.6                1.4               0.2

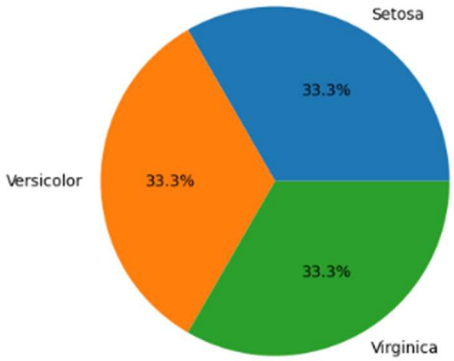
   Species
0        0
1        0
2        0
3        0
4        0
```











Result

The Iris dataset was successfully visualized using Matplotlib. Different graphs were generated to explore data distribution, feature relationships, and class composition. This experiment demonstrated the importance of Exploratory Data Analysis (EDA) in understanding datasets before building Machine Learning models.

LAB04

Date:

Title of the Lab: Machine Learning Life Cycle: Data Collection, Preprocessing, Training, Testing, and Prediction

Aim: To understand and implement the complete Machine Learning Life Cycle, including data collection, preprocessing, model training, testing, evaluation, and prediction using Python and Scikit-learn.

Software Requirements

- Python 3.x
- Jupyter Notebook / Google Colab
- NumPy
- Pandas
- Matplotlib
- Scikit-learn

Theory: The Machine Learning Life Cycle is a systematic process followed to develop, train, evaluate, deploy, and maintain a Machine Learning model. It ensures that the model is trained on quality data and provides accurate predictions. Stages of the Machine Learning Life Cycle:

i. Problem Definition

ii. Data Collection

iii. Data Preprocessing

iv. Train-Test Split

v. Model Selection

vi. Model Training

vii. Model Testing

viii. Model Evaluation

ix. Prediction

Algorithm

1. Import the required libraries.
2. Load the Iris dataset.
3. Create feature matrix (X) and target vector (y).
4. Split the dataset into training and testing sets.
5. Train the Decision Tree model.
6. Predict the testing data.
7. Calculate model accuracy.
8. Predict the class of a new flower sample.

#Python code

```
# Machine Learning Life Cycle

from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score

# Load Dataset
iris = load_iris()

X = iris.data
y = iris.target

print("Dataset Loaded Successfully")
print("Number of Samples:", len(X))

# Split Dataset
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)

print("\nTraining Samples:", len(X_train))
print("Testing Samples:", len(X_test))

# Train Model
model = DecisionTreeClassifier()

model.fit(X_train, y_train)

print("\nModel Training Completed")

# Prediction
y_pred = model.predict(X_test)

# Accuracy
accuracy = accuracy_score(y_test, y_pred)

print("\nModel Accuracy:", accuracy)

# Prediction for New Data
new_sample = [[5.1, 3.5, 1.4, 0.2]]

prediction = model.predict(new_sample)

print("\nPrediction for New Sample:")

print(iris.target_names[prediction][0])
```

#OUTPUT

```
Dataset Loaded Successfully
Number of Samples: 150

Training Samples: 120
Testing Samples: 30

Model Training Completed

Model Accuracy: 1.0

Prediction for New Sample:
setosa
```

Result

The Machine Learning Life Cycle was successfully implemented. The dataset was collected, preprocessed, divided into training and testing sets, used to train a Decision Tree Classifier, evaluated using accuracy, and finally used to predict the class of new data.

LAB05

Date:

Title of the Lab: Implementation of Probability Concepts and Bayes' Theorem using Python.

Aim: To understand and implement the basic concepts of **Probability** and **Bayes' Theorem** using Python for solving simple probabilistic problems related to Machine Learning.

Software Requirements

- Python 3.x
- Jupyter Notebook / Google Colab
- NumPy
- Pandas
- Matplotlib
- Scikit-learn

Theory

Probability

Probability is a branch of mathematics that measures the likelihood of an event occurring. It is represented by a value between **0 and 1**.

- **0** → Impossible Event
- **1** → Certain Event
- **0 < P < 1** → Possible Event

Formula :

P(A) = Number of Favorable Outcomes / Total Number of Outcomes

Example

For a fair die,

Sample Space: S= {1,2,3,4,5,6}

Probability of getting **4**: P (4) = 1/6

Algorithm

Part-A: Probability

1. Define the sample space.
2. Count the favorable outcomes.
3. Calculate probability using the probability formula.
4. Display the result.

Part-B: Bayes' Theorem

1. Define Prior Probability.
2. Define Likelihood.
3. Define Evidence Probability.
4. Apply Bayes' Theorem.
5. Display the Posterior Probability.

PYTHON CODE

```
#LAB 05
#Implementation of Probability Concepts and Bayes' Theorem using Python


print("----- Probability Example -----")

# Example: Probability of getting an even number on a die

sample_space = 6
favourable = 3

probability = favourable / sample_space

print("Probability of getting an even number =", probability)

# ------------------------------------------

print("\n----- Bayes Theorem Example -----")

# Medical Diagnosis Example

P_Disease = 0.01          # Prior Probability

P_Positive_given_Disease = 0.95     # Likelihood

P_Positive = 0.05         # Evidence

Posterior = (P_Positive_given_Disease * P_Disease) / P_Positive

print("Posterior Probability =", Posterior)
```

OUTPUT

```
----- Probability Example -----
Probability of getting an even number = 0.5

----- Bayes Theorem Example -----
Posterior Probability = 0.18999999999999997
```

Result

The concepts of **Probability** and **Bayes' Theorem** were successfully implemented using Python. The program correctly calculated the probability of an event and the posterior probability using Bayes' Theorem, demonstrating their importance in Machine Learning.

LAB06

Date:

Title of the Lab: Implementation of Gaussian Naïve Bayes Classifier on a Real Dataset

Aim: To implement the **Gaussian Naïve Bayes Classifier** using the **Iris dataset** and evaluate its performance for classification.

Software Requirements

- Python 3.x
- Jupyter Notebook / Google Colab
- NumPy
- Pandas
- Matplotlib
- Scikit-learn

Theory:

Naïve Bayes Classifier: The **Naïve Bayes Classifier** is a supervised Machine Learning algorithm used mainly for **classification problems**. It is based on **Bayes' Theorem** and assumes that all input features are **independent** of each other.

Despite this strong assumption, Naïve Bayes performs well on many real-world classification tasks.

Bayes' Theorem

The Naïve Bayes algorithm uses the following formula:

$$P(A|B) = \frac{P(B|A) \times P(A)}{P(B)}$$

Where:

- **P(A)** → Prior Probability
- **P(B|A)** → Likelihood
- **P(B)** → Evidence
- **P(A|B)** → Posterior Probability

Algorithm

1. Import the required libraries.
2. Load the Iris dataset.
3. Separate features and target labels.
4. Split the dataset into training and testing sets.
5. Create a Gaussian Naïve Bayes model.
6. Train the model using the training dataset.
7. Predict the labels for the testing dataset.
8. Calculate the model accuracy.
9. Predict the class of a new flower sample.

Python code

```
#LAB 06
#Implementation of Gaussian Naïve Bayes Classifier on a Real Dataset

from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

# Load Iris Dataset
iris = load_iris()

X = iris.data
y = iris.target

print("Dataset Loaded Successfully")

# Split Dataset into Training and Testing
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)

print("Training Samples :", len(X_train))
print("Testing Samples :", len(X_test))

# Create Gaussian Naïve Bayes Model
model = GaussianNB()

# Train Model
model.fit(X_train, y_train)

print("\nModel Trained Successfully")

# Predict Test Data
y_pred = model.predict(X_test)

# Accuracy
accuracy = accuracy_score(y_test, y_pred)

print("\nAccuracy :", accuracy)
```

```python
# Confusion Matrix
print("\nConfusion Matrix")
print(confusion_matrix(y_test, y_pred))

# Classification Report
print("\nClassification Report")
print(classification_report(y_test, y_pred))

# Predict New Sample
new_sample = [[5.1, 3.5, 1.4, 0.2]]

prediction = model.predict(new_sample)

print("\nPrediction for New Flower :")
print(iris.target_names[prediction][0])
```

OUTPUT:

```
Dataset Loaded Successfully
Training Samples : 120
Testing Samples : 30

Model Trained Successfully

Accuracy : 1.0

Confusion Matrix
[[10  0  0]
 [ 0  9  0]
 [ 0  0 11]]

Classification Report
              precision    recall  f1-score   support

           0       1.00      1.00      1.00        10
           1       1.00      1.00      1.00         9
           2       1.00      1.00      1.00        11

    accuracy                           1.00        30
   macro avg       1.00      1.00      1.00        30
weighted avg       1.00      1.00      1.00        30


Prediction for New Flower :
setosa
```

Result

The **Gaussian Naïve Bayes Classifier** was successfully implemented using the Iris dataset. The model was trained, tested, and evaluated with high classification accuracy. The experiment demonstrated the effectiveness of Gaussian Naïve Bayes for classifying continuous numerical data.

LAB 07

Date:

Title of the Lab: Data Preprocessing Techniques: Missing Values, Encoding, Feature Scaling and Train-Test Split

Aim: To implement basic data preprocessing techniques such as handling missing values, encoding categorical data, feature scaling, and splitting the dataset into training and testing data using Python.

Requirements

- Python 3.x
- Pandas
- NumPy
- Scikit-learn
- Jupyter Notebook / Google Colab

Theory

Data preprocessing is an important step in Machine Learning. Raw datasets may contain missing values, categorical variables, and features with different scales. Missing values can be replaced using statistical methods such as mean or mode. Categorical values can be converted into numerical values using encoding. Feature scaling brings numerical features to a comparable scale. Finally, the dataset is divided into training and testing sets to build and evaluate the model.

Algorithm

- Import the required libraries.
- Create a dataset containing numerical and categorical values.
- Identify and handle missing values.
- Encode the categorical variable.
- Apply StandardScaler to numerical features.

- Split the dataset into training and testing sets.
- Display the processed data.

#Code

```
# Lab 07
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

# Create dataset
data = {
    'Age': [20, 21, 22, None, 24, 25, 26, 27],
    'Salary': [25000, 30000, None, 35000, 40000, 45000, 50000, 55000],
    'Gender': ['Male', 'Female', 'Male', 'Female',
               None, 'Male', 'Female', 'Male'],
    'Experience': [1, 2, 3, 4, 5, None, 7, 8]
}

df = pd.DataFrame(data)

print("Original Dataset:")
print(df)

# Handle missing numerical values
df['Age'] = df['Age'].fillna(df['Age'].mean())
df['Salary'] = df['Salary'].fillna(df['Salary'].mean())
df['Experience'] = df['Experience'].fillna(df['Experience'].mean())

# Handle missing categorical value
df['Gender'] = df['Gender'].fillna(df['Gender'].mode()[0])

print("\nAfter Handling Missing Values:")
print(df)

# Encoding categorical variable
df['Gender'] = df['Gender'].map({'Male': 0, 'Female': 1})

print("\nAfter Encoding:")
print(df)
```

```
# Separate features
X = df[['Age', 'Salary', 'Gender', 'Experience']]

# Feature Scaling
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

X_scaled = pd.DataFrame(
    X_scaled,
    columns=X.columns
)

print("\nScaled Features:")
print(X_scaled)

# Train-Test Split
X_train, X_test = train_test_split(
    X_scaled,
    test_size=0.2,
    random_state=42
)

print("\nTraining Data:")
print(X_train)

print("\nTesting Data:")
print(X_test)
```

#output

Original Dataset:

	Age	Salary	Gender	Experience
0	20.0	25000.0	Male	1.0
1	21.0	30000.0	Female	2.0
2	22.0	NaN	Male	3.0
3	NaN	35000.0	Female	4.0
4	24.0	40000.0	None	5.0
5	25.0	45000.0	Male	NaN
6	26.0	50000.0	Female	7.0
7	27.0	55000.0	Male	8.0

After Handling Missing Values:

	Age	Salary	Gender	Experience
0	20.000000	25000.0	Male	1.000000
1	21.000000	30000.0	Female	2.000000
2	22.000000	40000.0	Male	3.000000
3	23.571429	35000.0	Female	4.000000
4	24.000000	40000.0	Male	5.000000
5	25.000000	45000.0	Male	4.285714
6	26.000000	50000.0	Female	7.000000
7	27.000000	55000.0	Male	8.000000

After Encoding:

	Age	Salary	Gender	Experience
0	20.000000	25000.0	0	1.000000
1	21.000000	30000.0	1	2.000000
2	22.000000	40000.0	0	3.000000
3	23.571429	35000.0	1	4.000000
4	24.000000	40000.0	0	5.000000
5	25.000000	45000.0	0	4.285714
6	26.000000	50000.0	1	7.000000
7	27.000000	55000.0	0	8.000000

Scaled Features:

	Age	Salary	Gender	Experience
0	-1.564028e+00	-1.603567	-0.774597	-1.480026
1	-1.126100e+00	-1.069045	1.290994	-1.029583
2	-6.881724e-01	0.000000	-0.774597	-0.579141
3	1.555832e-15	-0.534522	1.290994	-0.128698
4	1.876834e-01	0.000000	-0.774597	0.321745
5	6.256112e-01	0.534522	-0.774597	0.000000
6	1.063539e+00	1.069045	1.290994	1.222630
7	1.501467e+00	1.603567	-0.774597	1.673073

Training Data:

	Age	Salary	Gender	Experience
0	-1.564028e+00	-1.603567	-0.774597	-1.480026
7	1.501467e+00	1.603567	-0.774597	1.673073
2	-6.881724e-01	0.000000	-0.774597	-0.579141
4	1.876834e-01	0.000000	-0.774597	0.321745
3	1.555832e-15	-0.534522	1.290994	-0.128698
6	1.063539e+00	1.069045	1.290994	1.222630

Testing Data:

	Age	Salary	Gender	Experience
1	-1.126100	-1.069045	1.290994	-1.029583
5	0.625611	0.534522	-0.774597	0.000000

Result

Thus, missing values were handled, categorical data was encoded, features were scaled, and the dataset was successfully divided into training and testing sets.

LAB 08

Date:

Title of the Lab: Implementation of Simple Linear Regression for Prediction

Aim: To implement Simple Linear Regression using Python and predict the marks of a student based on the number of hours studied.

Theory: Simple Linear Regression is a supervised learning algorithm used to predict a continuous output using one independent variable. It represents the relationship between input and output using a straight line.

$$Y = mX + c$$

where $m$is the slope and $c$is the intercept.

Algorithm

- Import the required libraries.
- Create the dataset.
- Divide the data into training and testing sets.
- Create a Linear Regression model.
- Train the model.
- Predict the test values.
- Calculate the R^2 score.
- Predict marks for a new student.
- Plot the regression line.

#code

```
# Lab 08
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score

# Dataset
X = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10]).reshape(-1, 1)
y = np.array([35, 40, 45, 50, 55, 60, 65, 70, 75, 80])

# Train-Test Split
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42
)

# Create model
model = LinearRegression()

# Train model
model.fit(X_train, y_train)

# Prediction
y_pred = model.predict(X_test)

print("Coefficient:", model.coef_[0])
print("Intercept:", model.intercept_)

print("\nActual Values:")
print(y_test)

print("\nPredicted Values:")
print(y_pred)

# R2 Score
r2 = r2_score(y_test, y_pred)
print("\nR2 Score:", r2)
```

```
# R2 Score
r2 = r2_score(y_test, y_pred)
print("\nR2 Score:", r2)

# Predict for 7 hours
new_prediction = model.predict([[7]])
print("\nPredicted marks for 7 hours:", new_prediction[0])

# Plot
plt.scatter(X, y)
plt.plot(X, model.predict(X))
plt.xlabel("Study Hours")
plt.ylabel("Marks")
plt.title("Simple Linear Regression")
plt.show()
```

#output

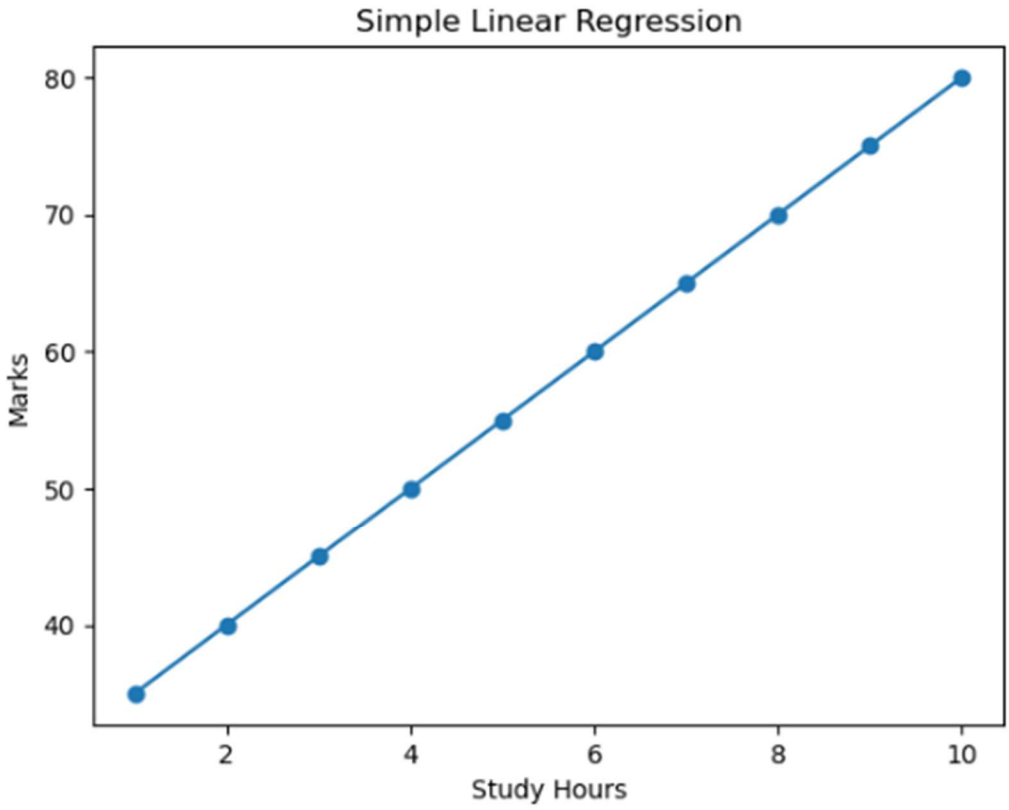
```
Coefficient: 5.000000000000001
Intercept: 29.999999999999996

Actual Values:
[75 40]

Predicted Values:
[75. 40.]

R2 Score: 1.0

Predicted marks for 7 hours: 65.0
```



Result

Thus, Simple Linear Regression was successfully implemented and used to predict student marks based on study hours.

LAB 09

Date:

Title of the Lab: Implementation of Multiple Linear Regression

Aim: To implement Multiple Linear Regression using multiple independent variables for predicting examination marks.

Theory: Multiple Linear Regression is an extension of Simple Linear Regression. It uses two or more independent variables to predict a continuous dependent variable.

$$Y = b_0 + b_1X_1 + b_2X_2 + \cdots + b_nX_n$$

For example, examination marks can be predicted using study hours, attendance, and assignment marks.

Algorithm

- Import the required libraries.
- Create the dataset.
- Select independent and dependent variables.
- Split the data into training and testing sets.
- Train the Multiple Linear Regression model.
- Predict the test values.
- Calculate R^2 score.
- Predict the output for a new student.

#Code

```python
# Lab 09
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score

# Dataset
data = np.array([
    [2, 70, 60, 45],
    [3, 75, 65, 50],
    [4, 80, 70, 55],
    [5, 85, 75, 62],
    [6, 90, 80, 70],
    [7, 92, 85, 76],
    [8, 95, 90, 82],
    [9, 96, 92, 88],
    [10, 98, 95, 94],
    [1, 65, 55, 40]
])

# Features
X = data[:, 0:3]

# Target
y = data[:, 3]

# Train-Test Split
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42
)

# Create model
model = LinearRegression()

# Train
model.fit(X_train, y_train)
```

```python
# Prediction
y_pred = model.predict(X_test)

print("Coefficients:")
print(model.coef_)

print("\nIntercept:")
print(model.intercept_)

print("\nActual Values:")
print(y_test)

print("\nPredicted Values:")
print(y_pred)

# R2 Score
r2 = r2_score(y_test, y_pred)

print("\nR2 Score:", r2)

# Predict for a new student
# Study Hours = 6
# Attendance = 88
# Assignment = 78

new_student = [[6, 88, 78]]

prediction = model.predict(new_student)

print("\nPredicted Marks:", prediction[0])
```

#output:

```
Coefficients:
[ 5.02055465 -0.38955954  0.56117455]

Intercept:
28.584665579119303

Actual Values:
[94 50]

Predicted Values:
[93.92495922 50.90570962]

R2 Score: 0.9991467551202824

Predicted Marks: 68.19836867862969
```

Result

Thus, Multiple Linear Regression was successfully implemented using multiple features to predict examination marks.

LAB 10

Date:

Title of the Lab: Implementation and Comparison of Polynomial Regression

Aim: To implement Polynomial Regression and compare its performance with Simple Linear Regression for nonlinear data.

Theory: Polynomial Regression is used when the relationship between input and output is nonlinear. It extends Linear Regression by adding powers of the input variable.

$$Y = b_0 + b_1X + b_2X^2 + \cdots + b_nX^n$$

The degree of the polynomial determines the complexity of the model.

Algorithm

- Create a nonlinear dataset.
- Train a Linear Regression model.
- Transform the features into polynomial features.
- Train the Polynomial Regression model.
- Calculate R^2 for both models.
- Compare their performance.
- Plot the results.

#code

```
#Lab 10
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import PolynomialFeatures
from sklearn.metrics import r2_score

# Non-linear dataset
X = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10]).reshape(-1, 1)

y = np.array([
    3, 7, 13, 21, 31,
    43, 57, 73, 91, 111
])

# -------------------------
# Linear Regression
# -------------------------

linear_model = LinearRegression()
linear_model.fit(X, y)

linear_pred = linear_model.predict(X)

linear_r2 = r2_score(y, linear_pred)

# -------------------------
# Polynomial Regression
# -------------------------

poly = PolynomialFeatures(degree=2)

X_poly = poly.fit_transform(X)

poly_model = LinearRegression()
poly_model.fit(X_poly, y)

poly_pred = poly_model.predict(X_poly)

poly_r2 = r2_score(y, poly_pred)
```

```
# Results
print("Linear Regression R2:", linear_r2)
print("Polynomial Regression R2:", poly_r2)

# Prediction
new_value = [[11]]

new_value_poly = poly.transform(new_value)

prediction = poly_model.predict(new_value_poly)

print("\nPolynomial Regression Prediction for X = 11:")
print(prediction[0])

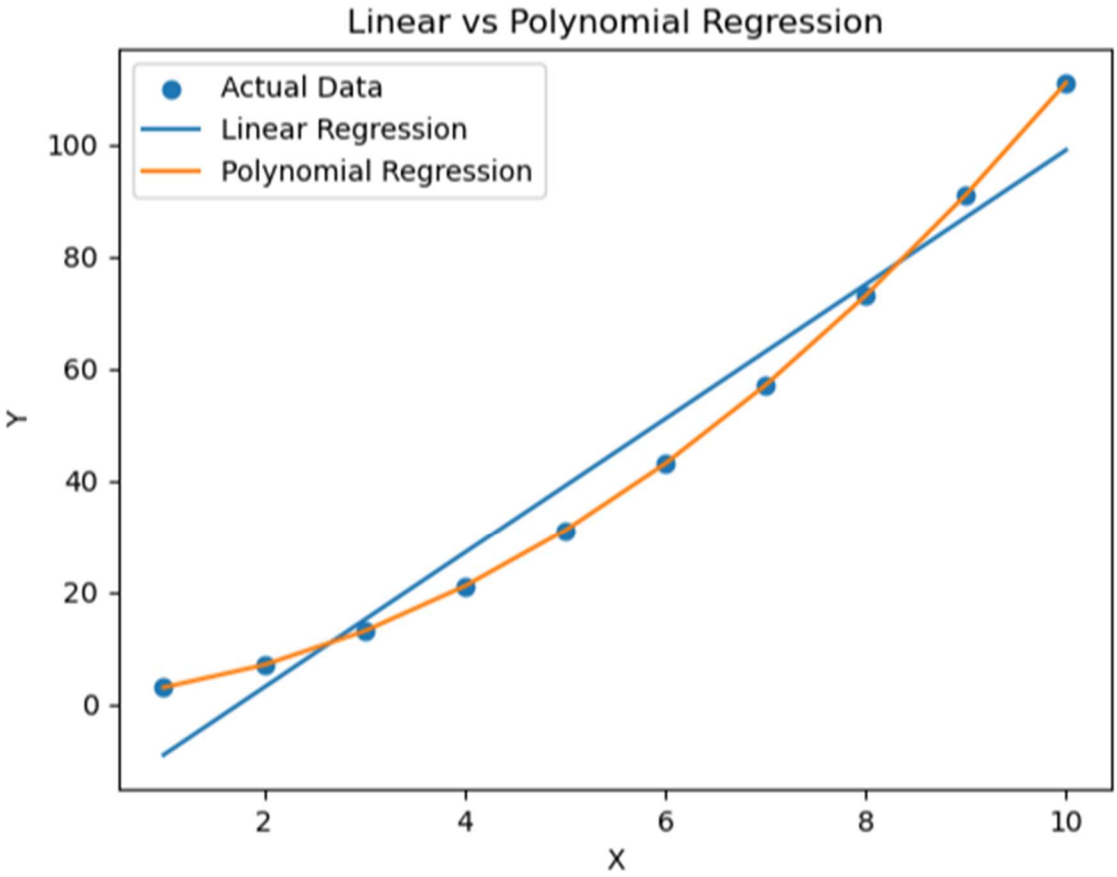
# Plot
plt.scatter(X, y, label="Actual Data")
plt.plot(X, linear_pred, label="Linear Regression")
plt.plot(X, poly_pred, label="Polynomial Regression")

plt.xlabel("X")
plt.ylabel("Y")
plt.title("Linear vs Polynomial Regression")
plt.legend()
plt.show()
```

#output

```
Linear Regression R2: 0.9574468085106383
Polynomial Regression R2: 1.0

Polynomial Regression Prediction for X = 11:
133.0
```



Result

Thus, Polynomial Regression was implemented successfully and compared with Linear Regression. Polynomial Regression provided a better representation of the nonlinear relationship.

LAB 11

Date:

Title of the Lab: Evaluation of Regression Models using MSE, RMSE, MAE and R^2 Score

Aim: To evaluate the performance of a regression model using MAE, MSE, RMSE, and R^2 score.

Theory: Regression evaluation metrics measure how closely the predicted values match the actual values.

MAE:

$$MAE = \frac{1}{n}\sum |\, y_i - \hat{y}_i \,|$$

MSE:

$$MSE = \frac{1}{n}\sum (y_i - \hat{y}_i)^2$$

RMSE:

$$RMSE = \sqrt{MSE}$$

R^2 Score**:**

$$R^2 = 1 - \frac{SS_{res}}{SS_{tot}}$$

Lower MAE, MSE and RMSE generally indicate better performance, while a higher R^2 indicates better fit.

3. Algorithm

- Create a regression dataset.
- Train a Linear Regression model.
- Generate predictions.
- Calculate MAE.
- Calculate MSE.
- Calculate RMSE.
- Calculate R^2 score.
- Compare the metrics.

#code

```
# Lab 11
import numpy as np
from sklearn.linear_model import LinearRegression
from sklearn.metrics import (
    mean_absolute_error,
    mean_squared_error,
    r2_score
)

# Dataset
X = np.array([1, 2, 3, 4, 5, 6]).reshape(-1, 1)

y = np.array([10, 20, 30, 40, 50, 60])

# Create and train model
model = LinearRegression()

model.fit(X, y)

# Prediction
y_pred = model.predict(X)

print("Actual Values:")
print(y)

print("\nPredicted Values:")
print(y_pred)
```

```python
# MAE
mae = mean_absolute_error(y, y_pred)

# MSE
mse = mean_squared_error(y, y_pred)

# RMSE
rmse = np.sqrt(mse)

# R2
r2 = r2_score(y, y_pred)

print("\nEvaluation Metrics")
print("-------------------------")
print("MAE :", mae)
print("MSE :", mse)
print("RMSE:", rmse)
print("R2  :", r2)
```

#Output

```
Actual Values:
[10 20 30 40 50 60]

Predicted Values:
[10. 20. 30. 40. 50. 60.]

Evaluation Metrics
-------------------------
MAE : 2.6645352591003757e-15
MSE : 1.5251310834272894e-29
RMSE: 3.905292669477269e-15
R2  : 1.0
```

Result: Thus, the regression model was successfully evaluated using MAE, MSE, RMSE, and R^2 score.

LAB 12

Date:

Title of the Lab:

Analysis of Overfitting and Underfitting in Regression Models

Aim: To analyze the effects of underfitting and overfitting using polynomial regression models of different degrees.

Theory: Underfitting occurs when a model is too simple to learn the underlying pattern in the data. It generally performs poorly on both training and testing data. Overfitting occurs when a model is too complex and learns noise along with the actual pattern. It may perform very well on training data but poorly on unseen testing data.

Algorithm

- Generate nonlinear data.
- Divide the data into training and testing sets.
- Create polynomial models with different degrees.
- Train each model.
- Calculate training and testing MSE.
- Compare the results.
- Identify underfitting and overfitting behavior.

#code

```
# Lab 12
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error

# Generate data
np.random.seed(42)

X = np.linspace(0, 10, 30).reshape(-1, 1)

y = 2 * X.ravel()**2 + 3 * X.ravel() + 5
y = y + np.random.normal(0, 20, size=len(y))

# Train-Test Split
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.3,
    random_state=42
)

degrees = [1, 2, 10]

for degree in degrees:

    # Polynomial transformation
    poly = PolynomialFeatures(degree=degree)

    X_train_poly = poly.fit_transform(X_train)
    X_test_poly = poly.transform(X_test)

    # Model
    model = LinearRegression()

    model.fit(X_train_poly, y_train)

    # Predictions
    train_pred = model.predict(X_train_poly)
    test_pred = model.predict(X_test_poly)
```

```
    # Predictions
    train_pred = model.predict(X_train_poly)
    test_pred = model.predict(X_test_poly)

    # Errors
    train_mse = mean_squared_error(y_train, train_pred)
    test_mse = mean_squared_error(y_test, test_pred)

    print("\nPolynomial Degree:", degree)
    print("Training MSE:", train_mse)
    print("Testing MSE :", test_mse)

# Visualization
plt.scatter(X, y, label="Data")

x_curve = np.linspace(0, 10, 100).reshape(-1, 1)

for degree in degrees:

    poly = PolynomialFeatures(degree=degree)

    X_train_poly = poly.fit_transform(X_train)

    model = LinearRegression()
    model.fit(X_train_poly, y_train)

    curve = model.predict(poly.transform(x_curve))

    plt.plot(
        x_curve,
        curve,
        label=f"Degree {degree}"
    )

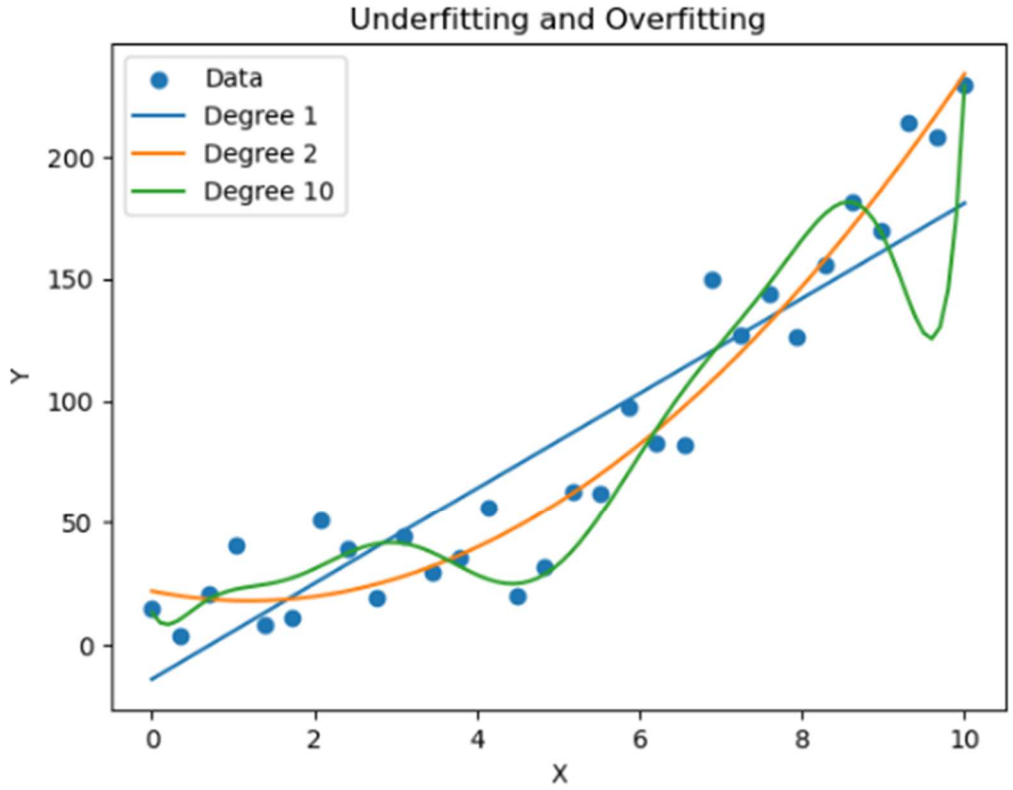
plt.xlabel("X")
plt.ylabel("Y")
plt.title("Underfitting and Overfitting")
plt.legend()
plt.show()
```

#Output

```
Polynomial Degree: 1
Training MSE: 780.1973698667713
Testing MSE : 525.3788379579994

Polynomial Degree: 2
Training MSE: 297.2241948707846
Testing MSE : 159.57694914177793

Polynomial Degree: 10
Training MSE: 142.6257824559255
Testing MSE : 1816.089125058463
```



Observation

- ✓ **Degree 1:** Model may be too simple → Underfitting.
- ✓ **Degree 2:** Suitable complexity → Better fit.
- ✓ **Degree 10:** Model may become excessively complex → Overfitting.

Result

Thus, underfitting and overfitting were successfully analyzed by comparing regression models with different polynomial degrees.